

A Concise Derivation of ISOMAP

Zhenwei Tang

Goal

Given n high-dimensional data points

$$x_i \in \mathbb{R}^p, \quad i = 1, \dots, n,$$

ISOMAP aims to find a low-dimensional embedding

$$z_i \in \mathbb{R}^k, \quad k \ll p,$$

such that the *geodesic* (manifold) distances between points are preserved:

$$\|z_i - z_j\|_2 \approx d_{\mathcal{M}}(x_i, x_j),$$

where $d_{\mathcal{M}}(x_i, x_j)$ denotes the geodesic distance along the underlying manifold $\mathcal{M}$.

The key insight is that Euclidean distances in the ambient space $\mathbb{R}^p$ can be misleading when the data lie on a curved manifold; geodesic distances faithfully reflect the intrinsic geometry. ISOMAP approximates geodesic distances via shortest paths on a neighbourhood graph, then recovers coordinates z_i by applying Classical Multidimensional Scaling (CMDS).

Step 1: Construct the Neighbourhood Graph

Why: Geodesic distances cannot be computed directly; we approximate them by "hopping" along nearby data points.

For each point x_i , connect it to its K nearest neighbours (or to all points within a ball of radius ε) using Euclidean distance. Assign edge weight

$$w_{ij} = \|x_i - x_j\|_2$$

to each edge (i, j) in the graph G . Let

$$W \in \mathbb{R}^{n \times n}, \quad W_{ij} = \begin{cases} \|x_i - x_j\|_2 & \text{if } (i, j) \in G, \\ \infty & \text{otherwise.} \end{cases}$$

Step 2: Compute All-Pairs Shortest Paths

Why: The geodesic distance between two points is the length of the shortest path along the manifold. On the graph, this corresponds to the shortest weighted path.

Using Dijkstra's algorithm (or Floyd–Warshall) on G , compute the geodesic-distance matrix

$$D_G \in \mathbb{R}^{n \times n}, \quad (D_G)_{ij} = d_G(x_i, x_j),$$

where $d_G(x_i, x_j)$ is the length of the shortest path between x_i and x_j in G . As $n \rightarrow \infty$ and K (or ε) grows appropriately, $d_G(x_i, x_j) \rightarrow d_{\mathcal{M}}(x_i, x_j)$.

Step 3: Convert Distances to Inner Products (Double Centering)

Why: Classical MDS (applied in Step 4) operates on an inner-product matrix, not a distance matrix. We use double centering to recover inner products from squared distances.

Let $\delta_{ij} = (D_G)_{ij}$ denote the estimated geodesic distance. Define the squared-distance matrix

$$\Delta \in \mathbb{R}^{n \times n}, \quad \Delta_{ij} = \delta_{ij}^2.$$

Applying the double-centering operator (see Appendix ??):

Define the centering matrix

$$H = I_n - \frac{1}{n} \mathbf{1} \mathbf{1}^T \in \mathbb{R}^{n \times n},$$

where I_n is the $n \times n$ identity matrix and $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector.

Compute the inner-product (Gram) matrix:

$$B = -\frac{1}{2} H \Delta H.$$

Why this works: If the embedding coordinates are $z_i \in \mathbb{R}^k$, then $\|z_i - z_j\|^2 = \|z_i\|^2 - 2z_i^T z_j + \|z_j\|^2$. After double centering (which removes the row/column means and thus eliminates the $\|z_i\|^2$ and $\|z_j\|^2$ terms), one recovers $B_{ij} = z_i^T z_j$, i.e. $B = ZZ^T$ where $Z = [z_1 | \dots | z_n]^T$.

Step 4: Eigendecomposition of B (Classical MDS)

Why: We want to find coordinates $Z \in \mathbb{R}^{n \times k}$ such that $ZZ^T = B$. This is a matrix factorization problem solved by eigendecomposition.

Applying eigendecomposition (see Appendix ??) to the symmetric matrix B :

$$B = V \Lambda V^T,$$

where

$$\begin{aligned} V &= [v_1 | v_2 | \dots | v_n] \in \mathbb{R}^{n \times n}, \quad \text{orthonormal eigenvectors,} \\ \Lambda &= \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n), \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n. \end{aligned}$$

Why B is positive semi-definite: If the geodesic distances are exactly Euclidean in some space, then $B = ZZ^T$ is a Gram matrix and hence PSD (see Appendix ??). In practice, B may have small negative eigenvalues due to approximation error; these are discarded.

Retain the top k non-negative eigenvalues $\lambda_1 \geq \dots \geq \lambda_k > 0$ and their eigenvectors $v_1, \dots, v_k$. Define

$$V_k = [v_1 | \dots | v_k] \in \mathbb{R}^{n \times k}, \quad \Lambda_k = \text{diag}(\lambda_1, \dots, \lambda_k) \in \mathbb{R}^{k \times k}.$$

Step 5: Recover the Low-Dimensional Embedding

Why: We need Z such that $ZZ^T \approx B$. Using the truncated eigendecomposition $B \approx V_k \Lambda_k V_k^T$, we set

$$Z = V_k \Lambda_k^{1/2},$$

where $\Lambda_k^{1/2} = \text{diag}(\sqrt{\lambda_1}, \dots, \sqrt{\lambda_k})$.

Verification:

$$ZZ^T = V_k \Lambda_k^{1/2} \left(V_k \Lambda_k^{1/2} \right)^T = V_k \Lambda_k^{1/2} \Lambda_k^{1/2} V_k^T = V_k \Lambda_k V_k^T \approx B. \quad \checkmark$$

The i -th row of Z gives the embedding coordinate:

$$z_i = \Lambda_k^{1/2} V_k^T e_i \in \mathbb{R}^k,$$

where e_i is the i -th standard basis vector (i.e. the i -th row of Z).

Conclusion

ISOMAP recovers the intrinsic low-dimensional geometry of data on a nonlinear manifold through three main stages:

1. **Graph construction:** approximate geodesic distances via shortest paths on the K -NN graph, obtaining D_G .
2. **Double centering:** convert squared geodesic distances to inner products via $B = -\frac{1}{2}H\Delta H$.
3. **Classical MDS:** recover embedding coordinates $Z = V_k \Lambda_k^{1/2}$ from the top- k eigenpairs of B .

The final embedding satisfies

$$\|z_i - z_j\|_2 \approx d_{\mathcal{M}}(x_i, x_j),$$

i.e. Euclidean distances in the low-dimensional space faithfully approximate geodesic distances on the original manifold.

ISOMAP embeds high-dimensional manifold data into $\mathbb{R}^k$ by replacing Euclidean distances with graph-approximated geodesic distances, then applying Classical MDS to recover coordinates that preserve intrinsic geometry.

A Double Centering

Let $z_1, \dots, z_n \in \mathbb{R}^k$ be centred embeddings, i.e. $\sum_i z_i = 0$. Define $\bar{z} = 0$. Then

$$\delta_{ij}^2 = \|z_i - z_j\|^2 = \|z_i\|^2 - 2z_i^T z_j + \|z_j\|^2.$$

Let $a_i = \|z_i\|^2$. Then

$$\begin{aligned} \frac{1}{n} \sum_j \delta_{ij}^2 &= a_i + \frac{1}{n} \sum_j a_j, \\ \frac{1}{n} \sum_i \delta_{ij}^2 &= a_j + \frac{1}{n} \sum_i a_i, \\ \frac{1}{n^2} \sum_{i,j} \delta_{ij}^2 &= \frac{2}{n} \sum_i a_i. \end{aligned}$$

The double-centering formula

$$B_{ij} = -\frac{1}{2} \left(\delta_{ij}^2 - \frac{1}{n} \sum_{j'} \delta_{ij'}^2 - \frac{1}{n} \sum_{i'} \delta_{i'j}^2 + \frac{1}{n^2} \sum_{i',j'} \delta_{i'j'}^2 \right)$$

simplifies to

$$B_{ij} = -\frac{1}{2} (-2z_i^T z_j) = z_i^T z_j,$$

i.e. $B = ZZ^T$. In matrix form, $B = -\frac{1}{2}H\Delta H$ where $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^T$ is the *centering matrix* satisfying $H^2 = H$ and $H\mathbf{1} = 0$.

B Eigendecomposition of a Symmetric Matrix

Spectral Theorem: For any real symmetric matrix $A = A^T \in \mathbb{R}^{n \times n}$, there exists an orthogonal matrix $V \in \mathbb{R}^{n \times n}$ (i.e. $V^T V = I$) and a diagonal matrix $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ with $\lambda_1 \geq \dots \geq \lambda_n$ such that

$$A = V\Lambda V^T.$$

The columns v_i of V are the *eigenvectors* of A , and λ_i are the corresponding *eigenvalues*, satisfying $Av_i = \lambda_i v_i$.

C Positive Semi-Definite (PSD) Matrix

A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is *positive semi-definite* (PSD), written $A \succeq 0$, if

$$u^T A u \geq 0 \quad \forall u \in \mathbb{R}^n.$$

Equivalently: all eigenvalues of A are non-negative, i.e. $\lambda_i \geq 0$ for all i .

Gram matrices are PSD: For any $Z \in \mathbb{R}^{n \times k}$, the matrix $B = ZZ^T$ satisfies

$$u^T B u = u^T Z Z^T u = \|Z^T u\|^2 \geq 0,$$

so $B \succeq 0$. This guarantees that, when geodesic distances are exactly Euclidean-embeddable, B has no negative eigenvalues and the ISOMAP embedding is exact.

D Dijkstra's Algorithm (Shortest Path)

Given a weighted graph $G = (V, E, w)$ with non-negative weights $w_{ij} \geq 0$, **Dijkstra's algorithm** computes the shortest-path distance from a source node s to all other nodes in $O(|E| + |V| \log |V|)$ time. Applied n times (once per source), it yields the full all-pairs shortest-path matrix D_G in $O(n(|E| + n \log n))$ time. Alternatively, **Floyd-Warshall** computes all pairs in $O(n^3)$ time, which is simpler to implement but less efficient for sparse graphs.